

Semantica formale di md2arxiv

Denotazionale e assiomatica

Fabio F.G. Buono

2026

Contents

1	Introduzione	2
2	Sintassi astratta	2
2.1	Il linguaggio sorgente	2
2.2	Il linguaggio target	3
3	Semantica denotazionale: livello base	3
3.1	Domini e funzioni di significato	3
3.2	La funzione di escape	4
3.3	Semantica degli inline	4
3.4	Semantica dei block	5
3.5	Semantica del corpo del documento	7
4	Semantica denotazionale: domini CPO e punto fisso	7
4.1	Motivazione	7
4.2	Ordini parziali completi	7
4.3	Il CPO delle stringhe con ordine prefisso	8
4.4	La concatenazione come funzione continua	8
4.5	La funzione $\mathcal{E}[\cdot]$ come minimo punto fisso	9
5	Semantica assiomatica	9
5.1	Il sistema di Hoare	9
5.2	La funzione <code>read_until</code>	10
5.3	La funzione <code>try_delim</code>	11
5.4	Invariante globale del lexer	12
6	Conclusioni	13
	Appendici	14
A	Monoidi liberi e linguaggi formali	14
B	Punto fisso minimo e teorema di Kleene	14

1 Introduzione

`md2arxiv` è un transpiler da Markdown accademico a $\text{\LaTeX}$ organizzato come una pipeline di compilazione in miniatura in cui un lexer analizza gli elementi di riga, un parser costruisce la struttura di blocco, un modulo di emissione traduce l'albero sintattico in testo $\text{\LaTeX}$ e un validatore controlla la coerenza delle citazioni bibliografiche. Nonostante la compattezza del progetto, ogni fase ha una semantica precisa che si presta a una trattazione formale, e questo documento si propone di fornirla secondo due approcci classici.

La semantica denotazionale assegna un significato matematico a ogni costrutto del linguaggio sorgente per induzione sulla struttura dell'albero sintattico ed è naturale per le funzioni pure di emissione, per lo stile ho cercato di seguire quello degli appunti del corso di linguaggi dell'università di Pisa. La semantica assiomatica invece descrive il comportamento delle funzioni con stato mediante precondizioni e postcondizioni collegate da regole di derivazione, questa è perfetta per le funzioni imperative del lexer che modificano un cursore e un buffer. Notiamo che i domini sintattici sono introdotti mediante grammatiche in forma BNF, i domini semantici sono definiti come ordini parziali completi e le funzioni di significato sono date per clausole sulla struttura sintattica.

2 Sintassi astratta

2.1 Il linguaggio sorgente

Per assegnare un significato a un programma è necessario descriverne con precisione la struttura sintattica, prescindendo dai dettagli concreti della rappresentazione testuale e mettendo in luce soltanto le relazioni strutturali tra i costrutti. Questa descrizione prende il nome di *sintassi astratta* e costituisce il punto di partenza di ogni trattazione semantica formale. Quindi il linguaggio sorgente accettato da `md2arxiv` è un sottoinsieme di Markdown e la sua sintassi astratta è organizzata su due livelli gerarchici che riflettono la struttura del modulo `ast.ml`: gli elementi *inline*, che vivono all'interno di una riga di testo e ne esprimono la formattazione locale, e i *block*, che costituiscono la struttura portante del documento a livello di paragrafo, sezione e ambiente.

Definizione 2.1 (Sintassi astratta degli inline). Il dominio sintattico **Inline** è definito dalla seguente grammatica in forma BNF, dove s, k, t e u denotano elementi di **String**.

$i ::=$	<code>Text(s)</code>	testo grezzo
	<code>Bold(s)</code>	grassetto
	<code>Italic(s)</code>	corsivo
	<code>Code(s)</code>	codice inline
	<code>Cite(k)</code>	citazione bibliografica
	<code>Link(t, u)</code>	collegamento ipertestuale

Una *sequenza di inline* è un elemento di **Inline**^{*}, l'insieme delle liste finite di elementi di **Inline**.

Il dominio **Block** descrive i costrutti strutturali del documento. Tra questi, **Heading** porta con sé il livello gerarchico come numero naturale, **UnorderedList** è una lista di sequenze di inline, **Table** separa esplicitamente l'intestazione dalle righe dati, e **Bibliography** è una lista di coppie chiave-descrizione che corrisponde direttamente all'ambiente `thebibliography` di $\text{\LaTeX}$.

Definizione 2.2 (Sintassi astratta dei block). Il dominio sintattico **Block** è definito dalla

seguente grammatica in forma BNF.

$b ::=$	$\text{Heading}(n, \mathbf{i})$	$n \in \mathbb{N}, n \geq 1, \mathbf{i} \in \mathbf{Inline}^*$
	$\text{Paragraph}(\mathbf{i})$	$\mathbf{i} \in \mathbf{Inline}^*$
	$\text{UnorderedList}(\mathbf{i}_1, \dots, \mathbf{i}_k)$	$\mathbf{i}_j \in \mathbf{Inline}^*, k \geq 0$
	$\text{CodeBlock}(s)$	$s \in \mathbf{String}$
	$\text{Image}(a, p)$	$a, p \in \mathbf{String}$
	$\text{Table}(H, R)$	$H \in \mathbf{String}^+, R \in (\mathbf{String}^*)^*$
	$\text{Bibliography}((k_1, d_1), \dots, (k_m, d_m))$	$k_j, d_j \in \mathbf{String}$

Un *documento* **Doc** è una coppia $(\mu, \mathbf{b})$ dove μ è un record di metadati e $\mathbf{b} \in \mathbf{Block}^*$ è la lista dei block del corpo.

Osservazione 2.1. Il costruttore $\text{Heading}(n, \mathbf{i})$ non pone un limite superiore su n , e la funzione di emissione mappa ogni $n \geq 3$ su `\subsubsection`. Il requisito $H \in \mathbf{String}^+$ per le tabelle garantisce che ogni tabella abbia almeno una colonna nell'intestazione, mentre l'insieme delle righe R può essere vuoto, producendo una tabella con sola intestazione.

2.2 Il linguaggio target

Il linguaggio target di `md2arxiv` è **String**, l'insieme di tutte le stringhe finite sull'alfabeto Unicode, privo di struttura sintattica propria. I frammenti $\text{\LaTeX}$ prodotti dalle funzioni di emissione sono stringhe che acquisiscono significato solo quando vengono interpretate dal motore tipografico, e la semantica denotazionale che costruiamo è una famiglia di funzioni dai domini sintattici strutturati verso questo dominio piatto, con la composizionalità come proprietà centrale.

3 Semantica denotazionale: livello base

3.1 Domini e funzioni di significato

L'approccio denotazionale consiste nell'associare a ogni frase sintattica un oggetto matematico, detto *denotazione*, che ne cattura il significato indipendentemente dalla struttura interna. Nel caso di `md2arxiv`, la denotazione di ogni costrutto è una stringa $\text{\LaTeX}$, e il significato di un inline è il frammento di testo che lo rappresenta, il significato di un block è l'ambiente $\text{\LaTeX}$ corrispondente, il significato di una lista di block è la loro concatenazione. La traduzione è *composizionale* nel senso che il significato di un termine dipende unicamente dal significato dei suoi sottotermini diretti, e la funzione di emissione non ha memoria né stato globale.

Il dominio semantico principale è **String** con la concatenazione $\cdot$, che forma un monoide $(\mathbf{String}, \cdot, \varepsilon)$ con elemento neutro la stringa vuota ε . Le funzioni di significato che costituiscono la semantica di `md2arxiv` sono le seguenti.

Definizione 3.1 (Funzioni di significato). La semantica denotazionale di `md2arxiv` è data dalla famiglia di funzioni

$$\begin{aligned} \mathcal{S} : \mathbf{String} &\rightarrow \mathbf{String}, & \mathcal{I}[\![\cdot]\!] : \mathbf{Inline} &\rightarrow \mathbf{String}, & \mathcal{I}^*[\![\cdot]\!] : \mathbf{Inline}^* &\rightarrow \mathbf{String}, \\ \mathcal{B}[\![\cdot]\!] : \mathbf{Block} &\rightarrow \mathbf{String}, & \mathcal{E}[\![\cdot]\!] : \mathbf{Block}^* &\rightarrow \mathbf{String}. \end{aligned}$$

3.2 La funzione di escape

Il testo grezzo che compare in un documento Markdown può contenere caratteri che in $\text{\LaTeX}$ hanno significato speciale, come $\&$, $\%$, $\$$ e altri. Prima di poter inserire una stringa arbitraria in un documento $\text{\LaTeX}$ occorre quindi proteggerla sostituendo ogni carattere speciale con la sequenza di escape corrispondente. Questa operazione è svolta dalla funzione $\mathcal{S}$, definita carattere per carattere come omomorfismo del monoide delle stringhe.

Definizione 3.2 (Funzione di escape). La funzione $\mathcal{S} : \mathbf{String} \rightarrow \mathbf{String}$ è definita estendendo per omomorfismo la seguente funzione sui singoli caratteri $c \in \Sigma$.

$$\mathcal{S}(c) = \begin{cases} \backslash\& & c = \& \\ \backslash\% & c = \% \\ \backslash\$ & c = \$ \\ \backslash\# & c = \# \\ \backslash_ & c = _ \\ \backslash\{ & c = \{ \\ \backslash\} & c = \} \\ \backslash\text{asciitilde}\{ & c = \sim \\ \backslash\text{asciicircum}\{ & c = ^ \\ \backslash\text{backslash}\{ & c = \backslash \\ c & \text{altrimenti} \end{cases}$$

Per una stringa $s = c_1c_2 \cdots c_n$ si ha quindi $\mathcal{S}(s) = \mathcal{S}(c_1) \cdot \mathcal{S}(c_2) \cdots \mathcal{S}(c_n)$, con $\mathcal{S}(\varepsilon) = \varepsilon$.

Proposizione 3.1. $\mathcal{S}$ è un omomorfismo del monoide $(\mathbf{String}, \cdot, \varepsilon)$ in sé stesso, cioè $\mathcal{S}(s \cdot t) = \mathcal{S}(s) \cdot \mathcal{S}(t)$ per ogni $s, t \in \mathbf{String}$.

Proof. La definizione carattere per carattere e l'estensione per concatenazione rendono la proprietà immediata per induzione sulla lunghezza di s . $\square$

3.3 Semantica degli inline

Il nucleo della semantica denotazionale è la funzione $\mathcal{I}[\![\cdot]\!]$, che assegna a ogni costruttore di **Inline** un frammento $\text{\LaTeX}$ per casi indipendenti l'uno dall'altro, riflettendo la composizionalità a livello di tipo di dato algebrico.

Definizione 3.3 (Funzione di significato degli inline). La funzione $\mathcal{I}[\![\cdot]\!] : \mathbf{Inline} \rightarrow \mathbf{String}$ è definita per casi sul costruttore.

$$\begin{aligned} \mathcal{I}[\![\text{Text}(s)]\!] &= \mathcal{S}(s) \\ \mathcal{I}[\![\text{Bold}(s)]\!] &= \backslash\text{textbf}\{ \cdot \mathcal{S}(s) \cdot \} \\ \mathcal{I}[\![\text{Italic}(s)]\!] &= \backslash\text{emph}\{ \cdot \mathcal{S}(s) \cdot \} \\ \mathcal{I}[\![\text{Code}(s)]\!] &= \backslash\text{texttt}\{ \cdot \mathcal{S}(s) \cdot \} \\ \mathcal{I}[\![\text{Cite}(k)]\!] &= \backslash\text{cite}\{ \cdot k \cdot \} \\ \mathcal{I}[\![\text{Link}(t, u)]\!] &= \backslash\text{href}\{ \cdot u \cdot \} \{ \cdot \mathcal{S}(t) \cdot \} \end{aligned}$$

Osservazione 3.1. Notiamo due asimmetrie nella definizione. In $\text{Cite}(k)$ la chiave k non viene sottoposta a $\mathcal{S}$, perché le chiavi bibliografiche sono identificatori $\text{\LaTeX}$ sintetici che non

contengono caratteri speciali per costruzione. In $\text{Link}(t, u)$ il testo t viene escapato ma l'URL u no, perché il pacchetto `hyperref` gestisce internamente i caratteri speciali negli URL e un ulteriore escape produrrebbe output malformato.

La funzione $\mathcal{I}^*[\cdot]$ estende $\mathcal{I}[\cdot]$ alle sequenze di inline per semplice concatenazione dei significati dei singoli elementi, che è la forma più elementare di composizionalità.

Definizione 3.4 (Semantica di una sequenza di inline). La funzione $\mathcal{I}^*[\cdot] : \mathbf{Inline}^* \rightarrow \mathbf{String}$ è definita per induzione strutturale sulla lista.

$$\mathcal{I}^*[\cdot] = \varepsilon \quad \text{e} \quad \mathcal{I}^*[i :: \mathbf{i}] = \mathcal{I}[i] \cdot \mathcal{I}^*[\mathbf{i}].$$

Esempio 3.1. Consideriamo la sequenza che rappresenta la frase “vedi [hoare1969].” nel documento sorgente, cioè $[\text{Text}(\text{"vedi "}), \text{Cite}(\text{"hoare1969"}), \text{Text}(\text{"."})]$. Applicando la definizione si ottiene:

$$\begin{aligned} \mathcal{I}^*[\text{Text}(\text{"vedi "}), \text{Cite}(\text{"hoare1969"}), \text{Text}(\text{"."})] \\ &= \mathcal{S}(\text{"vedi "}) \cdot \backslash\text{cite}\{\text{hoare1969}\} \cdot \mathcal{S}(\text{"."}) \\ &= \text{"vedi \text{cite}\{hoare1969\}."} \end{aligned}$$

3.4 Semantica dei block

La funzione $\mathcal{B}[\cdot]$ costituisce il livello più ricco della semantica denotazionale base. Ogni costruttore di **Block** corrisponde a un ambiente o a un comando $\text{\LaTeX}$, e la definizione per casi rispecchia fedelmente la struttura del codice OCaml in `emit.ml`. Per **Heading** e **Paragraph** i sottotermini sono sequenze di inline la cui semantica è già data da $\mathcal{I}^*[\cdot]$, per **UnorderedList** sono una lista di sequenze che vengono concatenate dopo aver preposto il marcatore `\item`, e per **Table** la definizione si appoggia su una funzione ausiliare di normalizzazione che compensi le eventuali incoerenze nel numero di colonne segnalate dal parser.

Definizione 3.5 (Funzione di significato dei block). La funzione $\mathcal{B}[\cdot] : \mathbf{Block} \rightarrow \mathbf{String}$ è definita per casi sul costruttore, con ogni caso che si appoggia su $\mathcal{I}^*[\cdot]$ per il trattamento dei sottotermini di tipo inline.

Heading. Il livello gerarchico determina il comando $\text{\LaTeX}$ tramite la funzione ausiliare $\text{cmd} : \mathbb{N} \rightarrow \mathbf{String}$ definita da $\text{cmd}(1) = \backslash\text{section}$, $\text{cmd}(2) = \backslash\text{subsection}$, e $\text{cmd}(n) = \backslash\text{subsubsection}$ per $n \geq 3$. Allora:

$$\mathcal{B}[\text{Heading}(n, \mathbf{i})] = \text{cmd}(n) \cdot \{ \cdot \mathcal{I}^*[\mathbf{i}] \cdot \} \cdot \backslash\mathbf{n}\backslash\mathbf{n}$$

Paragraph.

$$\mathcal{B}[\text{Paragraph}(\mathbf{i})] = \mathcal{I}^*[\mathbf{i}] \cdot \backslash\mathbf{n}\backslash\mathbf{n}$$

UnorderedList. Indichiamo con $\bigodot_{j=1}^k$ la concatenazione ordinata degli elementi di una sequenza indicizzata. Allora:

$$\begin{aligned} \mathcal{B}[\text{UnorderedList}(\mathbf{i}_1, \dots, \mathbf{i}_k)] &= \backslash\text{begin}\{\text{itemize}\}\backslash\mathbf{n} \\ &\quad \cdot \bigodot_{j=1}^k (\backslash\text{item} \cdot \mathcal{I}^*[\mathbf{i}_j] \cdot \backslash\mathbf{n}) \\ &\quad \cdot \backslash\text{end}\{\text{itemize}\}\backslash\mathbf{n}\backslash\mathbf{n} \end{aligned}$$

CodeBlock. Il contenuto s non viene sottoposto a $\mathcal{S}$ perché l'ambiente `verbatim` di $\text{\LaTeX}$ tratta il proprio contenuto come testo grezzo, senza interpretare sequenze di escape.

$$\mathcal{B}[\text{CodeBlock}(s)] = \text{\begin{verbatim}}\text{\n} \cdot s \cdot \text{\n}\text{\end{verbatim}}\text{\n}\text{\n}$$

Image.

$$\begin{aligned} \mathcal{B}[\text{Image}(a, p)] = & \text{\begin{figure}}[h]\text{\n} \\ & \cdot \text{\centering}\text{\n} \\ & \cdot \text{\includegraphics[width = 0.8\linewidth]{} \cdot p \cdot }\text{\n} \\ & \cdot \text{\caption{} \cdot \mathcal{S}(a) \cdot }\text{\n} \\ & \cdot \text{\end{figure}}\text{\n}\text{\n} \end{aligned}$$

Table. Sia $c = |H|$ il numero di colonne dell'intestazione. Poiché il parser può produrre righe con un numero di celle diverso da c segnalando un warning, la funzione di emissione normalizza ogni riga tramite $\nu_c : \text{String}^* \rightarrow \text{String}^c$, che porta ogni riga a c celle troncando le più lunghe e completando con ε le più corte.

$$\nu_c(r) = \begin{cases} (r_1, \dots, r_c) & |r| \geq c \\ (r_1, \dots, r_{|r|}, \underbrace{\varepsilon, \dots, \varepsilon}_{c-|r|}) & |r| < c \end{cases}$$

La funzione di riga, con $H = (h_1, \dots, h_c)$ e $R = (r_1, \dots, r_m)$, è definita da

$$\text{row}(s_1, \dots, s_c) = \mathcal{S}(s_1) \cdot \& \cdot \dots \cdot \& \cdot \mathcal{S}(s_c) \cdot \text{\textbackslash\textbackslash}\text{\n}.$$

Allora:

$$\begin{aligned} \mathcal{B}[\text{Table}(H, R)] = & \text{\begin{center}}\text{\n} \\ & \cdot \text{\begin{tabular}}{\underbrace{1 \dots 1}_c}\text{\n} \\ & \cdot \text{\toprule}\text{\n} \cdot \text{row}(H) \cdot \text{\midrule}\text{\n} \\ & \cdot \bigcirc_{j=1}^m \text{row}(\nu_c(r_j)) \\ & \cdot \text{\bottomrule}\text{\n} \cdot \text{\end{tabular}}\text{\n}\text{\end{center}}\text{\n}\text{\n} \end{aligned}$$

Bibliography.

$$\begin{aligned} \mathcal{B}[\text{Bibliography}((k_1, d_1), \dots, (k_m, d_m))] = & \text{\begin{thebibliography}}\{99\}\text{\n} \\ & \cdot \bigcirc_{j=1}^m (\text{\bibitem}{} \cdot k_j \cdot \text{\textbackslash} \cdot \mathcal{S}(d_j) \cdot \text{\n}) \\ & \cdot \text{\end{thebibliography}}\text{\n}\text{\n} \end{aligned}$$

3.5 Semantica del corpo del documento

La funzione $\mathcal{E}[\cdot]$ completa la semantica base occupandosi della lista di block che costituisce il corpo del documento. Il suo significato è la concatenazione dei significati dei singoli block nell'ordine in cui compaiono, e la sua definizione per induzione strutturale è la più diretta possibile.

Definizione 3.6 (Funzione di significato del corpo). La funzione $\mathcal{E}[\cdot] : \mathbf{Block}^* \rightarrow \mathbf{String}$ è definita per induzione strutturale sulla lista.

$$\mathcal{E}[\Box] = \varepsilon \quad \text{e} \quad \mathcal{E}[b :: \mathbf{b}] = \mathcal{B}[b] \cdot \mathcal{E}[\mathbf{b}].$$

Proposizione 3.2 (Composizionalità). *Le funzioni $\mathcal{I}[\cdot]$, $\mathcal{B}[\cdot]$ ed $\mathcal{E}[\cdot]$ sono composizionali nel senso che il significato di ogni termine è determinato unicamente dal suo costruttore e dal significato dei suoi sottotermini diretti, senza fare mai riferimento alla struttura interna di questi ultimi.*

Proof. La proprietà è diretta dalla struttura delle definizioni per casi, poiché ogni clausola invoca $\mathcal{I}[\cdot]$ o $\mathcal{B}[\cdot]$ esclusivamente sui sottotermini diretti del termine corrente, senza mai accedere alla struttura interna di questi. $\square$

4 Semantica denotazionale: domini CPO e punto fisso

4.1 Motivazione

Le funzioni definite nella sezione precedente sono totali poiché sia i tipi algebrici che le liste sono strutture finite e le funzioni di emissione le attraversano per ricorsione strutturale ben fondata. Detto questo notiamo che la teoria dei CPO, introdotta da Dana Scott e sviluppata sistematicamente da Winskel [3], fornisce il quadro matematico entro il quale la semantica denotazionale si estende in modo uniforme alle funzioni parziali, alla ricorsione generale e ai tipi infiniti. In questa sezione inseriamo le funzioni di emissione in questo quadro più generale, mostrando che $\mathcal{E}[\cdot]$ è il minimo punto fisso di un operatore continuo su un opportuno CPO di stringhe, e chiarendo in quale senso la ricorsione strutturale su liste sia un caso speciale della teoria generale del punto fisso.

4.2 Ordini parziali completi

Un ordine parziale completo, o CPO, è la struttura matematica minimale su cui è possibile interpretare la ricorsione come calcolo di un punto fisso. L'idea centrale è che gli elementi di un CPO rappresentano *approssimazioni* di un valore, con l'elemento minimo $\perp$ che rappresenta l'assenza totale di informazione, e l'ordine che cattura la nozione di "informazione maggiore". Una funzione ricorsiva è allora il limite di una catena di approssimazioni successive, e la continuità garantisce che questo limite esiste ed è esso stesso un punto fisso.

Definizione 4.1 (Ordine parziale e CPO). Un *ordine parziale* è una coppia $(D, \sqsubseteq)$ dove $\sqsubseteq$ è una relazione riflessiva, transitiva e antisimmetrica su D . Un *CPO* è un ordine parziale in cui ogni catena $d_0 \sqsubseteq d_1 \sqsubseteq d_2 \sqsubseteq \dots$ ha un limite superiore minimo $\bigsqcup_{n \geq 0} d_n \in D$.

Definizione 4.2 (CPO piatto). Dato un insieme A , il *CPO piatto* $A_\perp$ è ottenuto aggiungendo a A un elemento minimale $\perp$ con ordine $a \sqsubseteq b$ se e solo se $a = \perp$ oppure $a = b$. L'elemento $\perp$ rappresenta il risultato non ancora calcolato o indefinito.

Definizione 4.3 (Funzioni monotone e continue). Una funzione $f : D \rightarrow E$ tra CPO si dice *monotona* se $d \sqsubseteq d'$ implica $f(d) \sqsubseteq f(d')$, e *continua* se è monotona e preserva i limiti superiori di catene, nel senso che $f(\bigsqcup_n d_n) = \bigsqcup_n f(d_n)$ per ogni catena (d_n) .

Definizione 4.4 (Punto fisso minimo). Sia D un CPO con elemento minimo $\perp$ e sia $f : D \rightarrow D$ continua. Il *punto fisso minimo* di f è $\text{fix}(f) = \bigsqcup_{n \geq 0} f^n(\perp)$, dove $f^0(\perp) = \perp$ e $f^{n+1}(\perp) = f(f^n(\perp))$.

Proposizione 4.1 (Teorema di Knaster-Tarski). Nelle condizioni della definizione precedente, $\text{fix}(f)$ è il minimo punto fisso di f , nel senso che soddisfa $f(\text{fix}(f)) = \text{fix}(f)$ ed è il minimo elemento con questa proprietà.

Proof. La proprietà di punto fisso segue dalla continuità: $f(\bigsqcup_n f^n(\perp)) = \bigsqcup_n f^{n+1}(\perp) = \bigsqcup_n f^n(\perp)$. Se d è un altro punto fisso, allora $\perp \sqsubseteq d$ e per induzione $f^n(\perp) \sqsubseteq f^n(d) = d$ per ogni n , da cui $\text{fix}(f) = \bigsqcup_n f^n(\perp) \sqsubseteq d$. $\square$

4.3 Il CPO delle stringhe con ordine prefisso

Per applicare la teoria dei CPO alle funzioni di emissione occorre dotare **String** di una struttura di ordine parziale. La scelta più congeniale per le stringhe è l'ordine prefisso, in cui s è minore di t quando t si ottiene da s aggiungendo caratteri in coda; questo ordine cattura l'idea che una stringa parzialmente costruita è un'approssimazione della stringa completa, e che aggiungere caratteri equivale ad acquisire informazione.

Definizione 4.5 (Ordine prefisso su **String**_⊥). Sia **String**_⊥ = **String** $\cup$ { $\perp$ }. L'ordine $\sqsubseteq$ su **String**_⊥ è definito da $s \sqsubseteq t$ se e solo se $s = \perp$, oppure $s, t \in \mathbf{String}$ e $\exists u \in \mathbf{String}$ tale che $t = s \cdot u$.

Proposizione 4.2. (**String**_⊥, $\sqsubseteq$) è un CPO con elemento minimo $\perp$.

Proof. La riflessività segue da $s = s \cdot \varepsilon$. Per la transitività, se $t = s \cdot u$ e $r = t \cdot v$ allora $r = s \cdot (u \cdot v)$. Per l'antisimmetria, se $t = s \cdot u$ e $s = t \cdot v$ allora $s = s \cdot u \cdot v$, da cui $u = v = \varepsilon$ e $s = t$. Infine, le catene di stringhe finite sono stazionarie, quindi ogni catena ha limite superiore. $\square$

4.4 La concatenazione come funzione continua

Per poter usare $\cdot$ nella definizione del funzionale di $\mathcal{E}$ all'interno del CPO **String**_⊥, conviene estenderla in modo da gestire l'elemento $\perp$ e verificare che la funzione risultante sia continua.

Definizione 4.6 (Concatenazione stretta su **String**_⊥). La concatenazione stretta $\cdot_\perp : \mathbf{String}_\perp \times \mathbf{String}_\perp \rightarrow \mathbf{String}_\perp$ è definita da

$$s \cdot_\perp t = \begin{cases} \perp & \text{se } s = \perp \text{ oppure } t = \perp \\ s \cdot t & \text{altrimenti.} \end{cases}$$

Proposizione 4.3. Le funzioni $\lambda s. s \cdot_\perp t$ e $\lambda t. s \cdot_\perp t$ sono continue su (**String**_⊥, $\sqsubseteq$) per ogni valore fissato dell'altro argomento.

Proof. Le catene di stringhe definite in **String**_⊥ sono stazionarie: se (s_n) è una catena e $s_N \in \mathbf{String}$ per qualche N , allora $s_n = s_N$ per ogni $n \geq N$. La continuità segue dalla monotonicità e dalla stazionarietà delle catene. $\square$

4.5 La funzione $\mathcal{E}[\cdot]$ come minimo punto fisso

Il funzionale $\Phi : [\mathbf{Block}^* \rightarrow \mathbf{String}_\perp] \rightarrow [\mathbf{Block}^* \rightarrow \mathbf{String}_\perp]$ trasforma una funzione approssimata in una che sa trattare liste di un elemento in più, aggiungendo la capacità di elaborare la testa della lista e delegando il resto alla funzione argomento.

Definizione 4.7 (Funzionale di $\mathcal{E}$). Il funzionale Φ è definito ponendo, per ogni $f \in [\mathbf{Block}^* \rightarrow \mathbf{String}_\perp]$ e ogni lista $\mathbf{b}$,

$$\Phi(f)(\mathbf{b}) = \begin{cases} \varepsilon & \mathbf{b} = [] \\ \mathcal{B}_\perp[\![b]\!] \cdot_\perp f(\mathbf{b}') & \mathbf{b} = b :: \mathbf{b}' \end{cases}$$

dove $\mathcal{B}_\perp[\![\cdot]\!]$ è la versione di $\mathcal{B}[\![\cdot]\!]$ che usa $\cdot_\perp$ al posto di $\cdot$.

Proposizione 4.4. Φ è un operatore continuo sullo spazio $[\mathbf{Block}^* \rightarrow \mathbf{String}_\perp]$ ordinato puntualmente.

Proof. La monotonicità segue dalla monotonicità di $\cdot_\perp$: se $f \sqsubseteq g$ allora $\mathcal{B}_\perp[\![b]\!] \cdot_\perp f(\mathbf{b}') \sqsubseteq \mathcal{B}_\perp[\![b]\!] \cdot_\perp g(\mathbf{b}')$. La continuità è conseguenza della continuità di $\cdot_\perp$ e della struttura puntuale del limite nello spazio funzionale. $\square$

Proposizione 4.5 ($\mathcal{E}[\cdot]$ come minimo punto fisso). $\mathcal{E}[\cdot]$ è il minimo punto fisso di Φ , cioè $\mathcal{E}[\cdot] = \mathbf{fix}(\Phi) = \bigsqcup_{n \geq 0} \Phi^n(\lambda \mathbf{b}. \perp)$.

Proof. Sia $\perp_0 = \lambda \mathbf{b}. \perp$. Le prime approssimazioni sono:

$$\begin{aligned} \Phi^0(\perp_0)(\mathbf{b}) &= \perp \quad \text{per ogni } \mathbf{b} \\ \Phi^1(\perp_0)(\mathbf{b}) &= \begin{cases} \varepsilon & \mathbf{b} = [] \\ \perp & |\mathbf{b}| \geq 1 \end{cases} \\ \Phi^2(\perp_0)(\mathbf{b}) &= \begin{cases} \varepsilon & \mathbf{b} = [] \\ \mathcal{B}_\perp[\![b]\!] & \mathbf{b} = [b] \\ \perp & |\mathbf{b}| \geq 2 \end{cases} \end{aligned}$$

Per induzione su n , $\Phi^n(\perp_0)(\mathbf{b}) = \mathcal{E}[\![\mathbf{b}]\!]$ se $|\mathbf{b}| < n$ e $\perp$ se $|\mathbf{b}| \geq n$. Ogni $\mathcal{E}[\![\mathbf{b}]\!]$ è raggiunto in $|\mathbf{b}| + 1$ passi, e il limite della catena crescente $(\Phi^n(\perp_0))_{n \geq 0}$ è la funzione totale $\mathcal{E}[\![\cdot]\!]$, che per il Teorema di Knaster-Tarski è il minimo punto fisso di Φ . $\square$

Osservazione 4.1. Il fatto che $\mathcal{E}[\![\cdot]\!]$ sia già totale e che il punto fisso collassi su $\mathbf{Block}^* \rightarrow \mathbf{String} \subset \mathbf{String}_\perp$ è una conseguenza della struttura finita delle liste e della ricorsione strutturale ben fondata. La trattazione CPO aggiunge chiarezza concettuale, rendendo preciso il senso in cui le approssimazioni successive convergono alla funzione totale e mostrando che la ricorsione strutturale su liste è il caso limite della teoria generale, in cui il punto fisso è raggiunto in un numero finito di passi uguale alla lunghezza della lista più uno.

5 Semantica assiomatica

5.1 Il sistema di Hoare

La semantica assiomatica descrive il comportamento di un programma come una relazione tra stati prima e dopo l'esecuzione. L'approccio fu introdotto da Floyd [6] per i diagrammi

di flusso e formalizzato da Hoare [5] per i linguaggi di programmazione strutturati nella forma oggi universalmente adottata di *tripla di Hoare*

$$\{P\} C \{Q\},$$

dove P è la *precondizione*, C il comando e Q la *postcondizione*. La tripla esprime la *correttezza parziale*, nel senso che se la precondizione P vale prima dell'esecuzione di C e C termina, allora la postcondizione Q vale dopo. La correttezza totale richiede in aggiunta la dimostrazione che C termina sempre quando P è soddisfatta, ma per le funzioni che consideriamo la terminazione è ovvia dalla struttura del codice.

Applichiamo questo schema alle funzioni con stato del modulo `lexer.ml`. Lo stato rilevante è composto dal cursore i , un riferimento intero che indica la posizione corrente nella stringa di input s di lunghezza ℓ , e dal buffer di accumulo `buf`. Usiamo la notazione i_0 per il valore di i al momento dell'ingresso nella funzione, e $!i$ per il suo valore al momento dell'uscita.

5.2 La funzione `read_until`

La funzione `read_until` scansiona la stringa s a partire dalla posizione corrente del cursore e cerca la prima occorrenza del delimitatore `delim`, restituendo il testo compreso tra la posizione iniziale e il delimitatore trovato. In caso di mancato reperimento del delimitatore, solleva l'eccezione `Not_found` senza modificare il cursore, lasciando al chiamante la possibilità di reagire al fallimento senza perdere informazioni sulla posizione corrente.

```
let read_until delim =
  assert (String.length delim > 0);
  assert (!i >= 0 && !i <= len);
  let i_pre = !i in
  let dlen = String.length delim in
  let j = ref !i in
  while !j + dlen <= len && String.sub s !j dlen <> delim do
    incr j
  done;
  if !j + dlen > len then raise Not_found;
  let content = String.sub s !i (!j - !i) in
  i := !j + dlen;
  assert (!i > i_pre && !i <= len);
  content
```

Definizione 5.1 (Triple di Hoare per `read_until`). Sia $d = |\text{delim}|$ e $i_0 = !i$. La specifica formale di `read_until` è espressa da due triple, una per ciascun esito possibile.

Quando il delimitatore è presente a partire dalla posizione i_0 :

$$\begin{aligned} & \{ 0 \leq i_0 \leq \ell \wedge d > 0 \wedge \exists j \geq i_0. j + d \leq \ell \wedge s[j \dots j+d) = \text{delim} \} \\ & \quad \text{read_until } \text{delim} \\ & \{ \text{result} = s[i_0 \dots j_{\min}) \wedge !i = j_{\min} + d \wedge !i > i_0 \wedge !i \leq \ell \} \end{aligned}$$

dove $j_{\min}$ è il più piccolo $j \geq i_0$ tale che $s[j \dots j+d) = \text{delim}$.

Quando il delimitatore è assente:

$$\begin{aligned} & \{ 0 \leq i_0 \leq \ell \wedge d > 0 \wedge \forall j \geq i_0. j + d \leq \ell \Rightarrow s[j \dots j+d) \neq \text{delim} \} \\ & \quad \text{read_until } \text{delim} \\ & \{ \text{Not_found sollevato} \wedge !i = i_0 \} \end{aligned}$$

Osservazione 5.1. Tra le due postcondizioni quella del caso di fallimento è la più sottile, perché non è immediatamente leggibile dal codice. Il cursore i rimane invariato a i_0 perché l'unica istruzione che lo modifica, cioè $i := !j + dlen$, si trova dopo il controllo `if !j + dlen > len` che in caso di fallimento provoca l'uscita anticipata tramite `raise Not_found`, rendendo quella riga irraggiungibile. Questa proprietà è irrinunciabile per la correttezza di `try_delim`, che dipende dal fatto che un tentativo fallito di `read_until` non lasci tracce sullo stato.

Proposizione 5.1 (Correttezza parziale di `read_until`). *La funzione `read_until` soddisfa le due triple di Hoare enunciate.*

Proof. L'invariante del ciclo `while` è la proposizione

$$I(j) \equiv i_0 \leq j \leq \ell \wedge \forall k. i_0 \leq k < j \Rightarrow s[k \dots k+d) \neq \text{delim},$$

che vale vacuamente all'ingresso con $j = i_0$ e viene preservata dal corpo del ciclo, il quale incrementa j solo quando $s[j \dots j+d) \neq \text{delim}$.

Caso successo. Il ciclo si arresta con $s[j \dots j+d) = \text{delim}$, e per $I(j)$ questo j è il primo tale indice $\geq i_0$, cioè $j = j_{\min}$. La funzione restituisce $s[i_0 \dots j_{\min})$ e pone $!i = j_{\min} + d$. Da $j_{\min} \geq i_0$ e $d \geq 1$ segue $!i > i_0$; da $j_{\min} + d \leq \ell$ segue $!i \leq \ell$.

Caso fallimento. Il ciclo si arresta con $j + d > \ell$, e per $I(j)$ nessuna occorrenza di `delim` esiste in $s[i_0 \dots \ell-d]$. La funzione solleva `Not_found` senza aver mai assegnato a i , quindi $!i = i_0$. $\square$

5.3 La funzione `try_delim`

La funzione `try_delim` gestisce l'intera sequenza di operazioni necessaria per riconoscere un costrutto delimitato, incluso il recupero in caso di fallimento. Quando il delimitatore di chiusura non viene trovato, il testo del delimitatore di apertura viene reintegrato nel buffer come testo letterale in modo da non perdere caratteri di input, e la condizione $|fallback_str| = \text{open_len}$ presente nel codice come `assert` garantisce che questo reintegro sia sempre corretto.

```
let try_delim open_len delim make fallback_str =
  assert (String.length fallback_str = open_len);
  flush ();
  i := !i + open_len;
  (try acc := make (read_until delim) :: !acc
   with Not_found -> Buffer.add_string buf fallback_str)
```

Definizione 5.2 (Triple di Hoare per `try_delim`). Siano $i_0 = !i$, $ol = \text{open_len}$, $fb = \text{fallback_str}$ con $|fb| = ol$, e acc_0 il contenuto dell'accumulatore prima dell'esecuzione di `flush()`. La preconditione comune alle due triple è

$$P \equiv 0 \leq i_0 \leq \ell \wedge ol \geq 1 \wedge |fb| = ol \wedge i_0 + ol \leq \ell,$$

mentre le postcondizioni distinguono i due casi. Quando il delimitatore di chiusura è presente a partire da $i_0 + ol$:

$$\{P\} \text{ try_delim } \{ acc = \text{make}(\text{content}) :: acc_0 \wedge !i > i_0 \wedge buf = \varepsilon \}$$

Quando il delimitatore di chiusura è assente a partire da $i_0 + ol$:

$$\{P\} \text{ try_delim } \{ buf = fb \wedge !i = i_0 + ol \wedge acc = acc_0 \}$$

Osservazione 5.2. La struttura della postcondizione nel caso di fallimento L'istruzione $i := i + \text{open_len}$, che porta il cursore oltre i caratteri del delimitatore di apertura, viene eseguita incondizionatamente prima di qualsiasi tentativo, cosicché il cursore avanza di ol in entrambi gli esiti. Nel caso di successo questo avanzamento si somma a quello prodotto dalla lettura del contenuto e del delimitatore di chiusura, e il tutto è rappresentato nell'accumulatore dal nodo costruito da `make`. Nel caso di fallimento l'avanzamento rimane scoperto e viene compensato aggiungendo al buffer la stringa fb di lunghezza ol , ed è questa la ragione per cui la condizione $|fb| = ol$, presente nel codice come `assert`, non è una guardia di routine ma il cardine dell'invariante globale che garantisce la conservazione di ogni carattere dell'input.

Proposizione 5.2 (Correttezza parziale di `try_delim`). *La funzione `try_delim` soddisfa le due triple enunciate, assumendo che `read_until` soddisfi la propria specifica.*

Proof. L'esecuzione di `flush()` svuota il buffer trasferendo il suo contenuto in `acc`, dopodiché l'istruzione $i := i_0 + ol$ porta il cursore all'inizio del presunto contenuto delimitato e viene invocata `read_until delim`.

Caso successo. Per la specifica di `read_until`, la chiamata restituisce $content = s[i_0 + ol \dots j_{\min})$ e porta il cursore a $j_{\min} + d$, che supera $i_0 + ol$ e quindi i_0 , la funzione `make` viene applicata al contenuto e il risultato viene preposto ad `acc`, mentre il buffer rimane vuoto.

Caso fallimento. Per la specifica di `read_until` nel caso di mancato reperimento, il cursore non viene modificato e rimane a $i_0 + ol$, l'eccezione viene catturata, fb viene aggiunto al buffer e l'accumulatore rimane acc_0 . $\square$

5.4 Invariante globale del lexer

Le due triple di Hoare si compongono in un invariante globale che vale all'inizio e alla fine di ogni iterazione del ciclo principale di `Lexer.parse` e che costituisce la garanzia di correttezza più importante dell'intero modulo, nel senso che ogni carattere della stringa di input contribuisce una volta sola all'output accumulato.

Definizione 5.3 (Invariante globale del lexer). Ora indichiamo con $emitted(acc, buf)$ la stringa ottenuta concatenando nell'ordine corretto i testi di tutti gli inline in `acc` e il contenuto corrente del buffer `buf`; l'invariante globale è allora la proposizione

$$Inv \equiv 0 \leq i \leq \ell \wedge s[0 \dots i) = emitted(acc, buf),$$

che afferma che il prefisso di s già consumato dal cursore coincide con ciò che è stato prodotto nell'accumulatore e nel buffer.

Proposizione 5.3. *Inv è un invariante del ciclo principale di `Lexer.parse`, nel senso che è preservato da ogni ramo del ciclo.*

Proof. Per i costrutti gestiti da `try_delim`, la postcondizione garantisce che nel caso di successo il cursore avanza di $ol + |content| + d$ mentre all'accumulatore viene aggiunto un elemento il cui testo corrisponde a quei caratteri, e nel caso di fallimento il cursore avanza di ol mentre ol caratteri vengono aggiunti al buffer, cosicché in entrambe le situazioni $emitted(acc, buf)$ cresce quanto il cursore avanza, preservando Inv .

Per il caso del link a due fasi, il ramo più delicato è quello in cui la parentesi di apertura (viene trovata ma la chiusura) non compare prima della fine della stringa: in questo caso la postcondizione di `read_until` assicura che il cursore si ferma immediatamente dopo (, e

il reintegro tramite `String.sub s start (!i - start)` inserisce nel buffer i caratteri compresi tra `[]` e la posizione corrente, compensando l'avanzamento e preservando *Inv*.

Per il carattere ordinario, un singolo carattere viene aggiunto al buffer e il cursore incrementa di uno, il che rende la verifica di *Inv* immediata. □

6 Conclusioni

Speriamo che questo lavoro possa essere utile a qualcuno, si rimanda alla bibliografia per approfondimenti.

Appendici

A Monoidi liberi e linguaggi formali

Il dominio semantico **String** usato nel corpo di questo documento è l'insieme di tutte le stringhe finite su un alfabeto Σ , questa struttura va chiarita per capire perché la concatenazione con elemento neutro ε sia la struttura giusta per modellare il testo prodotto da un compilatore.

Quindi dato che un linguaggio formale su un alfabeto A è un sottoinsieme di A^* , e la teoria dei linguaggi formali può essere intesa come lo studio dei sottoinsiemi del monoide libero su A , e la chiusura di Kleene A^* è appunto il monoide libero su A : ogni linguaggio regolare su A è un sottoinsieme di A^* chiuso sotto unione, prodotto e generazione di sottomonoide [7]. Nel documento, **String** = Σ^* dove Σ è l'alfabeto Unicode e la funzione S è un omomorfismo del monoide libero (**String**, $\cdot, \varepsilon$) in sé stesso, e le funzioni di significato $\mathcal{I}[\cdot]$, $\mathcal{B}[\cdot]$, $\mathcal{E}[\cdot]$ sono tutte omomorfismi dal dominio sintattico strutturato verso questo monoide, il che garantisce che la semantica rispetti la struttura algebrica della concatenazione.

B Punto fisso minimo e teorema di Kleene

Nel corpo del documento si usa il Teorema di Knaster-Tarski per garantire l'esistenza del minimo punto fisso del funzionale Φ , notiamo che è utile precisare la distinzione tra due risultati spesso citati insieme. Il Teorema di Knaster-Tarski afferma che ogni funzione monotona su un reticolo completo ha un minimo punto fisso e questo risultato non dice nulla sul modo in cui tale punto fisso può essere calcolato, né richiede la continuità della funzione. Il Teorema di Kleene, o teorema del punto fisso per CPO, afferma qualcosa di più costruttivo: dato D un CPO con elemento minimo $\perp$ e $f : D \rightarrow D$ una funzione *continua* nel senso di Scott, cioè monotona e che preserva i limiti superiori di catene dirette, allora il minimo punto fisso di f è il limite della catena

$$\perp \sqsubseteq f(\perp) \sqsubseteq f^2(\perp) \sqsubseteq \dots$$

cioè $\text{fix}(f) = \bigsqcup_{n \geq 0} f^n(\perp)$ [8]. Notiamo che questa è la versione applicata nel documento, dato che la continuità di Φ permette di calcolare $\mathcal{E}[\cdot]$ come limite delle approssimazioni $\Phi^n(\perp_0)$, ciascuna delle quali è definita su liste di lunghezza crescente e la continuità è la condizione che trasforma un risultato di esistenza in un calcolo per approssimazioni successive.

References

- [1] R. Barbuti, P. Mancarella, F. Turini. *Elementi di Semantica Operazionale*. Lecture notes for Fondamenti di Programmazione, B.Sc. in Computer Science, University of Pisa, A.A. 2004/05. <https://pages.di.unipi.it/mancarella/FP/materiale/nuovasemop.pdf>
- [2] G. D. Plotkin. *A structural approach to operational semantics*. Technical Report DAIMI FN-19, Aarhus University, 1981. Reprinted in *J. Log. Algebr. Program.*, 60–61:17–139, 2004.
- [3] G. Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, Cambridge, MA, 1993. ISBN 0-262-73103-7. <https://direct.mit.edu/books/monograph/4338/The-Formal-Semantics-of-Programming-LanguagesAn>

- [4] P. Mancarella. *Sintassi dei Linguaggi di Programmazione*. Dispensa per il corso di Fondamenti di Programmazione, Università di Pisa. <https://pages.di.unipi.it/mancarella/FP/materiale/syntax.pdf>
- [5] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [6] R. W. Floyd. Assigning meanings to programs. *Proceedings of a Symposium on Applied Mathematics*, 19:19–32, 1967.
- [7] Wikipedia contributors. *Free monoid*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Free_monoid
- [8] Wikipedia contributors. *Kleene fixed-point theorem*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Kleene_fixed-point_theorem